

Proyecto Perritos

Aplicación de Registro e Interacción de Mascotas

Miranda Rodriguez Cruz

2026

Índice

1. Introducción	3
2. Objetivo General	3
3. Objetivos Específicos	3
4. Tecnologías Utilizadas	3
5. Capturas del Sistema	4
5.1. Pantalla de Login	4
5.2. Registro de Mascotas	4
5.3. Comentarios y Reacciones	4
6. Principios SOLID Aplicados	5
6.1. Single Responsibility Principle (SRP)	5
6.2. Open Closed Principle (OCP)	6
6.3. Liskov Substitution Principle (LSP)	6
6.4. Interface Segregation Principle (ISP)	7
6.5. Dependency Inversion Principle (DIP)	7
7. Patrones de Diseño Utilizados	7
7.1. Patrón MVC	7
7.1.1. Model	8
7.1.2. View	8
7.1.3. Controller	9
7.2. Component-Based Architecture	9
7.3. State Management	10
7.4. Service Pattern	10
7.5. Singleton	10
8. Integración con Cloudinary	10
9. Despliegue en Render	11
10. Generación APK	11

11.Conclusiones	11
12.Referencias	11

1. Introducción

En este proyecto desarrollé una aplicación llamada **Perritos**, utilizando Flutter y Dart, con el objetivo de crear una pequeña red social enfocada en mascotas.

La aplicación permite registrar perritos, agregar imágenes, comentar publicaciones y reaccionar mediante diferentes emojis. Además, integré Cloudinary para almacenar imágenes en la nube y Render para desplegar la aplicación web.

Durante el desarrollo del proyecto me enfoqué principalmente en mantener una estructura organizada y reutilizable mediante el uso de patrones de diseño y principios SOLID.

Una de las partes más importantes del proyecto fue separar correctamente responsabilidades para evitar código repetido y facilitar el mantenimiento del sistema.

2. Objetivo General

Desarrollar una aplicación multiplataforma que permita registrar mascotas e interactuar mediante comentarios y reacciones utilizando tecnologías modernas y servicios en la nube.

3. Objetivos Específicos

- Implementar autenticación de usuarios.
- Registrar mascotas dentro de la aplicación.
- Permitir comentarios y reacciones.
- Subir imágenes mediante Cloudinary.
- Aplicar patrones de diseño.
- Aplicar principios SOLID.
- Desplegar la aplicación utilizando Render.
- Generar un APK funcional.

4. Tecnologías Utilizadas

- Flutter
- Dart
- Cloudinary
- Render
- GitHub
- Visual Studio Code
- Overleaf

5. Capturas del Sistema

5.1. Pantalla de Login

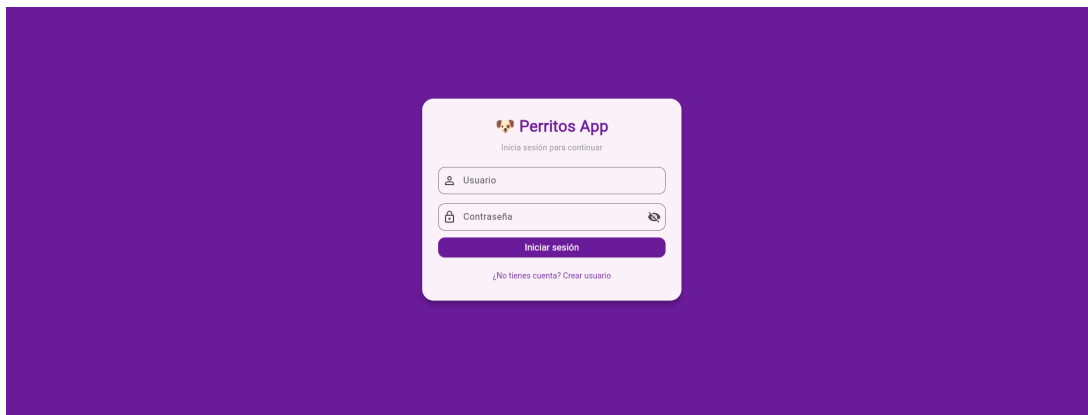


Figura 1: Pantalla de inicio de sesión

5.2. Registro de Mascotas

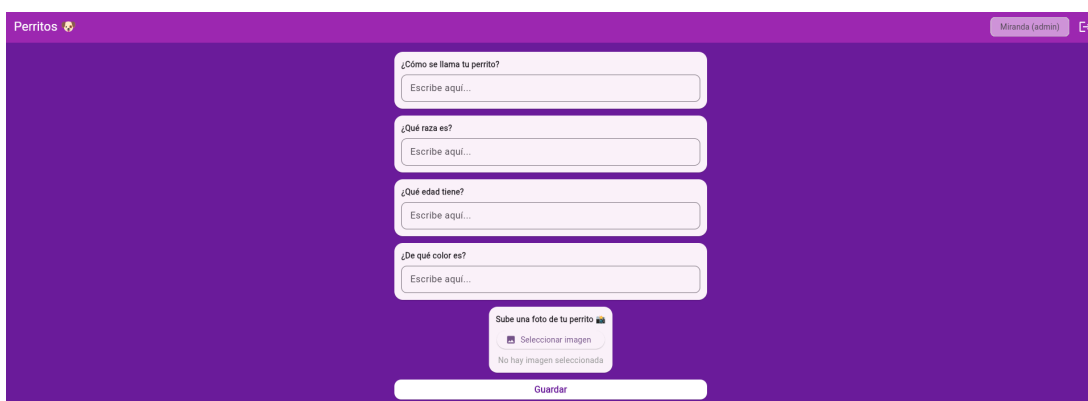


Figura 2: Formulario de registro de mascotas

5.3. Comentarios y Reacciones

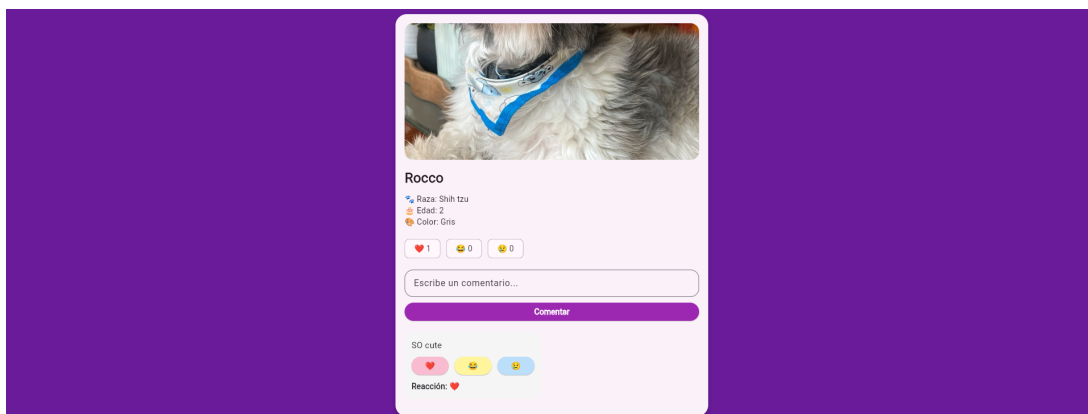


Figura 3: Sistema de comentarios y reacciones

6. Principios SOLID Aplicados

Durante el desarrollo del proyecto apliqué algunos principios SOLID con el objetivo de mantener el código más organizado, reutilizable y fácil de mantener.

Principio	Aplicación en el Proyecto
Single Responsibility Principle (SRP)	Cada clase fue creada con una responsabilidad específica.
Open Closed Principle (OCP)	El sistema puede extenderse agregando nuevas funcionalidades sin modificar completamente la estructura principal.
Liskov Substitution Principle (LSP)	Los widgets reutilizables pueden sustituirse manteniendo compatibilidad visual y funcional.
Interface Segregation Principle (ISP)	Los servicios fueron separados según sus responsabilidades.
Dependency Inversion Principle (DIP)	Las pantallas dependen de servicios especializados para manejar funcionalidades externas.

Cuadro 1: Principios SOLID utilizados

6.1. Single Responsibility Principle (SRP)

Este principio establece que una clase debe tener únicamente una responsabilidad.

En mi proyecto decidí separar completamente la lógica relacionada con Cloudinary dentro de un servicio independiente.

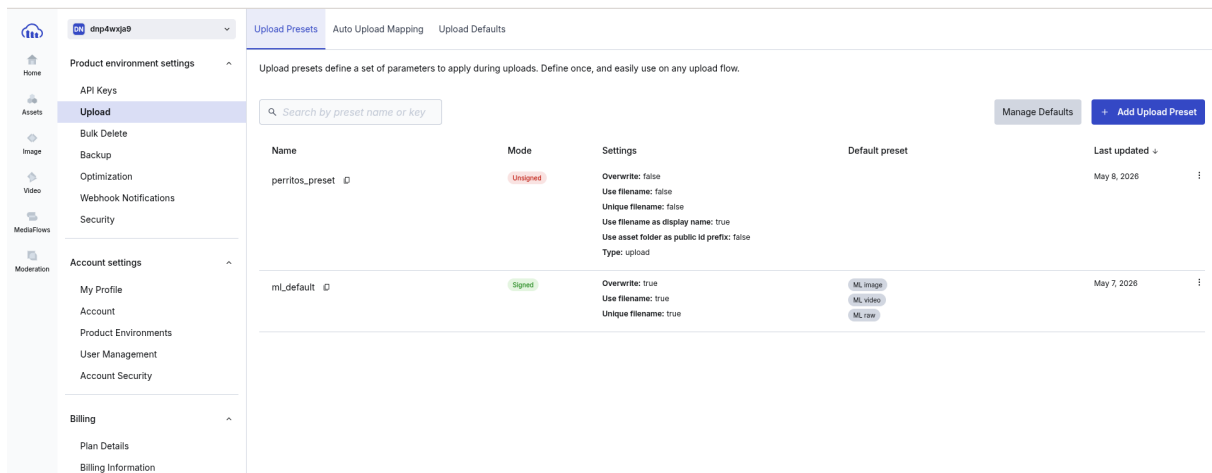


Figura 4: Servicio Cloudinary

```
1 class CloudinaryService {
2
3     static Future<String?> uploadImageWeb(
4         Uint8List imageBytes,
5     ) async {
6
7     }
8 }
```

Explicación

La clase `CloudinaryService` únicamente se encarga de subir imágenes a Cloudinary.

No contiene lógica relacionada con comentarios, mascotas o interfaz gráfica.

Esto me permitió mantener una mejor organización y evitar mezclar responsabilidades.

6.2. Open Closed Principle (OCP)

Este principio establece que el sistema debe estar abierto para extensión pero cerrado para modificación.

En el proyecto implementé las reacciones utilizando botones independientes, lo cual permite agregar nuevas reacciones fácilmente sin modificar completamente la lógica principal.

```
1 Wrap(  
2   spacing: 10,  
3   children: [  
4  
5     ElevatedButton(  
6       child: Text('      '),  
7     ),  
8  
9     ElevatedButton(  
10      child: Text('      '),  
11    ),  
12  
13    ElevatedButton(  
14      child: Text('      '),  
15    ),  
16  ],  
17 )
```

Explicación

Gracias a esta estructura puedo agregar nuevas reacciones posteriormente sin afectar otras partes del sistema.

Esto facilita la escalabilidad y mantenimiento de la aplicación.

6.3. Liskov Substitution Principle (LSP)

Este principio establece que los componentes deben poder sustituirse sin afectar el funcionamiento general del sistema.

Flutter trabaja naturalmente mediante widgets reutilizables.

```
1 Widget _tarjetaCampo(  
2   String titulo,  
3   TextEditingController controller,  
4 )
```

Explicación

En este caso el widget puede reutilizarse múltiples veces dentro de la interfaz manteniendo el mismo comportamiento y estructura visual.

Esto me permitió reducir código repetido y mantener consistencia en el diseño.

6.4. Interface Segregation Principle (ISP)

Este principio establece que las clases no deben depender de funcionalidades que no utilizan.

En el proyecto decidí separar los servicios según sus funciones específicas.

```
1 final _auth = AuthService();  
2  
3 final _picker = ImagePicker();
```

Explicación

Cada servicio tiene una responsabilidad independiente.

AuthService se encarga de autenticación mientras que ImagePicker únicamente maneja selección de imágenes.

Esto mejora la modularidad y organización del proyecto.

6.5. Dependency Inversion Principle (DIP)

Este principio establece que las clases principales deben depender de abstracciones y servicios especializados.

```
1 final imageUrl =  
2     await CloudinaryService.uploadImageWeb(  
3         imageBytes,  
4     );
```

Explicación

La pantalla principal no implementa directamente toda la lógica HTTP para subir imágenes.

En su lugar, delega esa responsabilidad a CloudinaryService.

Esto facilita reutilizar el servicio y mantener una mejor separación de responsabilidades.

7. Patrones de Diseño Utilizados

Durante el desarrollo del proyecto utilicé diferentes patrones de diseño con el objetivo de mantener una estructura organizada, modular y fácil de mantener.

Cada patrón fue utilizado para resolver problemas específicos dentro de la aplicación.

7.1. Patrón MVC

El patrón MVC fue uno de los patrones principales utilizados en el proyecto.

MVC significa:

- Model
- View
- Controller

La finalidad de este patrón es separar los datos, la interfaz gráfica y la lógica de control. Esto ayuda a mantener una estructura mucho más organizada.

7.1.1. Model

El Model representa los datos de la aplicación.

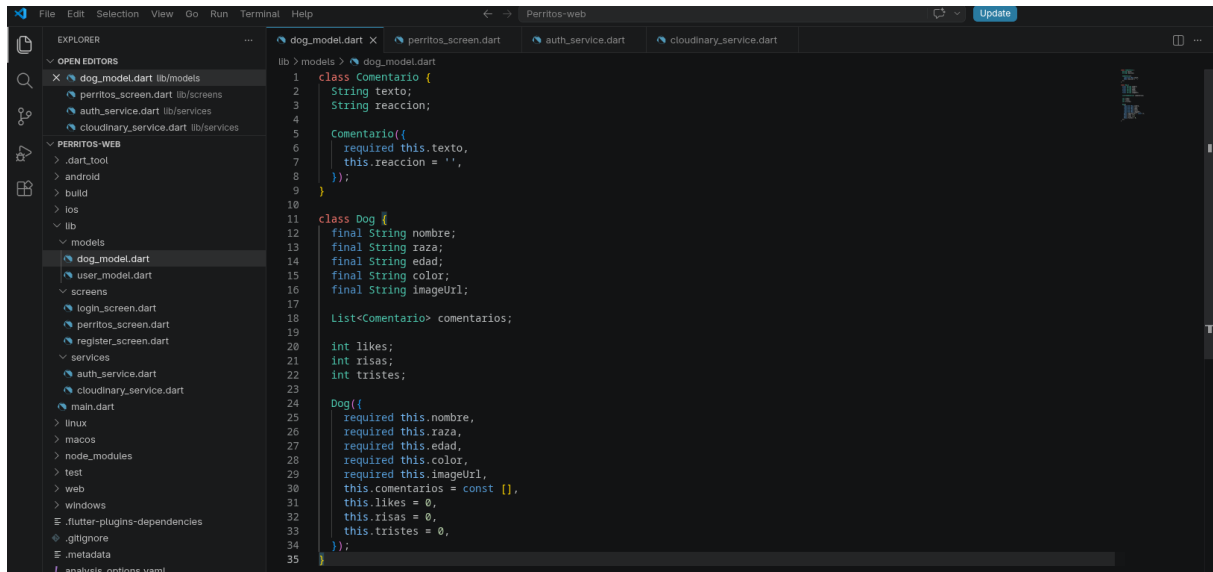


Figura 5: Modelo Dog y Comentario

```
1 class Dog {
2
3   final String nombre;
4   final String raza;
5   final String edad;
6   final String color;
7
8   List<Comentario> comentarios;
9 }
```

Explicación

La clase Dog representa la estructura principal de datos utilizada para almacenar información de cada mascota.

Además, contiene una lista de comentarios relacionados con cada publicación.

7.1.2. View

La View representa la interfaz gráfica mostrada al usuario.

```
1 class PerritosScreen
2 extends StatefulWidget {
3
4   const PerritosScreen({
5     super.key
6   });
7 }
```

Explicación

PerritosScreen representa la pantalla principal donde el usuario puede visualizar publicaciones, comentarios y reacciones.

7.1.3. Controller

El Controller se encarga de manejar eventos y actualizar la aplicación.

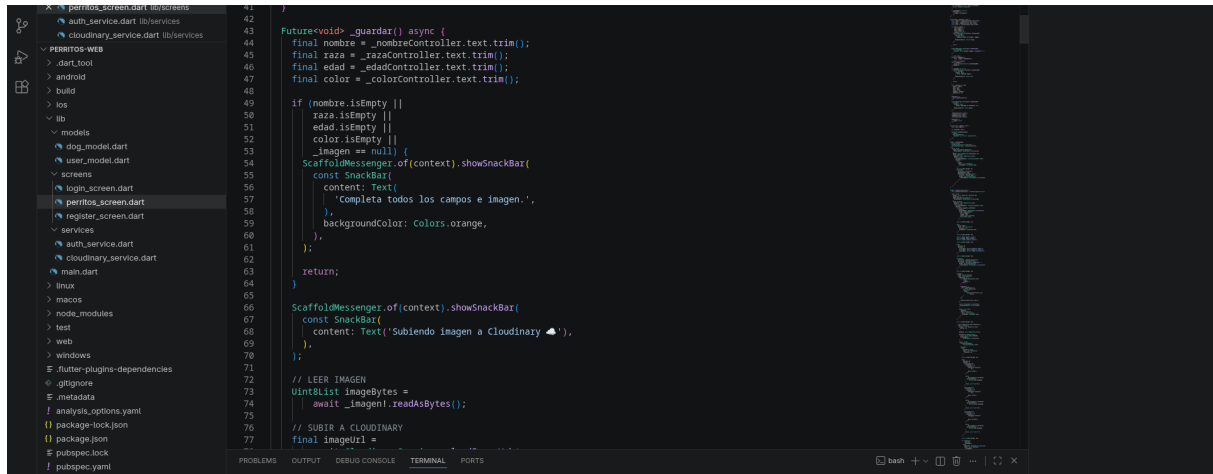


Figura 6: Lógica de PerritosScreen

```
1 Future<void> _guardar() async {
2
3   final nuevoPerro = Dog(
4     nombre: nombre,
5     raza: raza,
6     edad: edad,
7     color: color,
8     imageUrl: imageUrl,
9   );
10
11   setState(() {
12     perros.add(nuevoPerro);
13   });
14 }
```

Explicación

La función *guardar()* controla el proceso de registro de mascotas.

Aquí se crean objetos Dog y posteriormente se actualiza el estado de la interfaz utilizando *setState()*.

7.2. Component-Based Architecture

Flutter trabaja utilizando componentes reutilizables llamados widgets.

Durante el proyecto utilicé componentes personalizados para evitar repetir código.

```
1 Widget _tarjetaCampo(
2   String titulo,
3   TextEditingController controller,
4 )
```

Explicación

Este componente fue utilizado múltiples veces dentro del formulario principal.

Gracias a esto pude mantener una interfaz más consistente y reducir código repetido.

7.3. State Management

Para manejar cambios dinámicos dentro de la aplicación utilicé `setState()`.

```
1 setState(() {  
2   perros.add(nuevoPerro);  
3 });
```

Explicación

`setState()` actualiza automáticamente la interfaz cuando se agregan nuevas mascotas, comentarios o reacciones.

Esto permitió mantener una interacción dinámica dentro de la aplicación.

7.4. Service Pattern

También utilicé el patrón Service para separar funcionalidades externas dentro de servicios independientes.

```
1 class CloudinaryService {  
2  
3 }
```

Explicación

`CloudinaryService` encapsula toda la lógica relacionada con Cloudinary.

Esto permitió reutilizar fácilmente el servicio y mantener una mejor organización.

7.5. Singleton

El patrón Singleton busca garantizar que exista una única instancia compartida de un servicio.

```
1 final _auth = AuthService();
```

Explicación

En el proyecto reutilicé una única instancia de `AuthService` para manejar autenticación dentro de diferentes pantallas.

Esto evita crear múltiples instancias innecesarias y mejora la administración global del servicio.

8. Integración con Cloudinary

Cloudinary fue utilizado para almacenar imágenes en la nube.

El flujo implementado fue el siguiente:

1. El usuario selecciona una imagen.
2. Flutter envía la imagen a Cloudinary.
3. Cloudinary devuelve una URL.
4. La URL se guarda dentro del objeto Dog.

```
1 final imageUrl =  
2   await CloudinaryService.uploadImageWeb(  
3     imageBytes,  
4   );
```

9. Despliegue en Render

La aplicación fue desplegada utilizando Render como plataforma de hosting web.

Para realizar el despliegue primero actualicé el proyecto en GitHub y posteriormente configuré un Static Site dentro de Render.

10. Generación APK

Finalmente generé un APK funcional utilizando Flutter.

```
1 flutter build apk
```

11. Conclusiones

Durante el desarrollo de este proyecto comprendí la importancia de mantener una estructura organizada y modular.

La implementación de patrones de diseño y principios SOLID me permitió separar responsabilidades, reutilizar código y facilitar el mantenimiento del sistema.

Además, integrar tecnologías como Cloudinary y Render permitió acercar el proyecto a un entorno más realista y profesional.

12. Referencias

- Flutter Documentation
- Dart Documentation
- Cloudinary Documentation
- Render Documentation
- SOLID Principles